

Sistemas de gestión de rutinas personalizadas.

Autores:

Zaid Isamit Alvial 21.668.839-2

Tomás Contreras Hammad 21.064.979-4

Cristóbal Magnata Veragua 21.712.167-1

Nrc:

13046

Profesor:

Juan Calderon Maureira

Fecha de entrega:

28705/2026

1. INTRODUCCIÓN

El diseño manual de rutinas deportivas en centros de entrenamiento suele presentar problemas operativos crónicos, tales como errores en el cálculo de tiempos asignados o la repetición involuntaria de actividades en periodos consecutivos. La automatización de estos flujos mediante software mitiga las equivocaciones conceptuales del personal y optimiza significativamente la dosificación de las cargas físicas. Por ello, el objetivo principal de este trabajo es desarrollar una aplicación funcional con interfaz gráfica en Java Swing que administre de forma automatizada e inteligente la gestión de entrenamientos. Para lograrlo, el sistema implementa los pilares avanzados de la Programación Orientada a Objetos y una separación rígida entre la lógica de negocio y las interfaces visuales. El presente documento expone los detalles de la arquitectura adoptada, las decisiones de diseño aplicadas, el diagrama de clases del modelo y las conclusiones obtenidas del proceso.

2. DESCRIPCIÓN DE LA SOLUCIÓN

2.1. Arquitectura y Modelo de Dominio

El software se estructuró bajo una arquitectura desacoplada y modular dividida estrictamente en dos paquetes denominados backend y frontend. La capa del backend gestiona la lógica y persistencia temporal del sistema a través de colecciones de memoria de tipo `Vector<Ejercicio>`. La clase abstracta `Ejercicio` actúa como la plantilla madre del dominio, definiendo variables globales como código, nombre, tipo, intensidad, tiempo y última semana de uso. A partir de ella se desprenden mediante herencia simple las clases especialistas `EjercicioCardio` y `EjercicioFuerza`, las cuales automatizan la asignación de su tipo correspondiente. Por otro lado, la clase controladora `SistemaRutinas` administra el comportamiento general, ejecutando el algoritmo de filtro cruzado que valida la intensidad, las cuotas por categoría y la restricción matemática que impide repetir ejercicios usados en semanas consecutivas. Al arrancar el programa, la clase `Main` coordina la lectura automática del archivo plano `data/ejercicios.csv` sin necesidad de intervención del usuario.

2.2. Justificación de Decisiones de Diseño

La separación de responsabilidades y la estabilidad del sistema se resolvieron mediante la implementación de patrones de diseño específicos y lógicas de control en las interfaces. En primer lugar, se incorporó el patrón creacional Factory Method a través de EjercicioFactory, delegando la creación de los objetos de fuerza o cardio a una fábrica centralizada en lugar de sobrecargar al lector de archivos. Para la comunicación entre paquetes se utilizó el patrón Observer mediante la interfaz SubscriptorRutina; de esta forma, cuando el backend actualiza sus vectores de datos, notifica de manera asincrónica a la VentanaPrincipal sin acoplar código visual de Swing dentro de las clases de lógica pura.

El control de errores se implementó mediante el uso de excepciones de negocio personalizadas denominadas ArchivoInexistenteException y FormatoInvalidoException. Estas clases permiten capturar anomalías físicas o estructurales en el CSV (como datos numéricos corruptos o columnas incompletas) y desplegarlas de forma amigable a través de paneles de alerta interactivos. Finalmente, se programaron restricciones de frontera en la interfaz de usuario: la VentanaGeneracion ajusta dinámicamente los límites máximos de los componentes JSpinner según el stock real calculado por el backend para impedir que el usuario solicite más ejercicios de los disponibles. En esa misma línea, la VentanaRevision controla el estado de navegación bloqueando el botón "Anterior" en el primer elemento y mutando el botón "Siguiente" a "Ver Resumen" al llegar al último ejercicio de la lista.

2.3. Diagrama de Clases de la Solución



3. CONCLUSIÓN

El desarrollo de este sistema permitió consolidar de forma práctica la importancia de separar la interfaz gráfica de la lógica central del negocio mediante una arquitectura organizada por paquetes. A través del proceso, se lograron cumplir con éxito todos los objetivos estipulados por la cátedra, implementando herencia polimórfica, persistencia transitoria segura y un control robusto del flujo mediante excepciones y eventos estructurados. Como aprendizaje fundamental de ingeniería, la correcta aplicación de patrones de comportamiento como Observer demuestra ser una herramienta indispensable para garantizar que las aplicaciones de software sean modulares, limpias, fáciles de mantener y escalables ante futuros cambios en los requerimientos.